Group 67 Final Report: Tennis Match Winner Predictions

Martin Matlok, Leiah Nay, Nic Poulidis
{matlokm,nayl,poulidin}@mcmaster.ca

1 Introduction

We aim to predict the outcomes of professional men's singles tennis matches using historical performance statistics, player-specific data, and contextual match data. The outcome of any sport often appears unpredictable, yet all sports are driven by large amounts of data in an attempt to understand patterns of performance and surroundings. Tennis, with its rich statistical history and player-centric nature, provides an ideal scenario for leveraging machine learning to predict the outcome of games. Using a publicly available ATP dataset, which includes tournament and match data, as well as player statistics scraped from the ATP Tour website, we will build a model trained on features such as surface-specific win rates, serve and return performance, recent form, head-to-head history, and more. We will look to leverage machine learning to identify patterns in player performance that correspond to match outcomes and determine which features are most predictive of success.

2 Related Work

Many have attempted to use solve tennis game prediction with machine learning. Our idea was first inspired by Code (2025), who created a youtube video solving this issue. He opted to use decision trees, which we avoided because it was out of scope of the course content. A paper with a similar premise was found, Rui et al. (2024), which used XGBoost (A gradient boosted decision tree) to achieve an accuracy of 71%. We shifted our research towards a methodology within the scope of the course, with a goal to achieve similarly accurate results with a less popular method. We then found Seranno (2020), which attempted to solve tennis game prediction with an accuracy of 68.2% using neural networks. This is a masters thesis paper, so we took inspiration from it both for our choice of model and our accuracy goal of near 68.2%. We also discovered a github repo, Milkpainter (2025), which used an 8 model ensemble system to achieve an accuracy of 72.3%. All eight models are rather complex, so we chose to take inspiration from the Seranno (2020) paper's methodology instead. However, this repo gave us the highest accuracy we have discovered, giving us the upper limit of our goal for our model. All previous sources were analyzed to find possible features, but a different source, Lei et al. (2024) made a large impact as it had interesting insights into "momentum" as a factor in tennis matches. While mid-match analysis would be out of the scope of our model, this inspired features that took into account recent player performance. While this would add work to our pre-processing, requiring active calculations of each players stats up until a certain date, we were convinced this addition of "momentum" based features would improve our accuracy.

Further research post-progress report gave us new avenues for experimentation. A new upper limit of accuracy was discovered in Luo et al. (2025). An astounding 81% was accomplished using deep learning. This further re-enforces our idea to use a neural network, but recent lessons from class had us re-consider our choice of network. We originally chose a CNN, naively assuming that it was a more modern neural network that has more potential towards achieving a high accuracy. On learning that CNNs are usually for images, we re-read Seranno (2020). Seranno states he used a fully connected feed forward network, a simpler approach. Given that his paper achieved a higher accuracy than we were able to with our CNN, we decided to try a fully connected feed forward network, as opposed to improving our CNN. This is further supported by de Araujo Fernandes (2017), who uses a feed forward neural network as part of a 3 model voting system. Consistently, out of the three models, then neural network provided the best individual accuracy. This inspired us to

attempt an ensemble system to boost our accuracy.

3 Dataset

3.1 Scraping

The main dataset we used did not have some of the data needed (DataHub (n.d.)). Specifically, it was missing information regarding player statistics. To fill these gaps, we scraped the following features directly from ATP (ATP Tour (n.d.)):

- Date of Birth
- Height
- Weight
- Handedness (Right or Left Handed and Two-Handed or One-Handed Backhand)
- Country
- Surface Specific Statistics
  - Aces
  - Double Faults
  - 1st Serve
  - 1st Serve Points Won
  - 2nd Serve Points Won
  - Break Points Faced
  - Break Points Saved
  - Service Games Played
  - Service Games Won
  - Total Service Points Won
  - 1st Serve Return Points Won
  - 2nd Serve Return Points Won
  - Break Points Opportunities
  - Break Points Converted
  - Return Games Played
  - Return Games Won
  - Return Points Won
  - Total Points Won

Unfortunately, the scraper from the original git repository did not work, nor did it include surface specific statistics (DataHub (n.d.)). Therefore, an entirely new scraper was developed. JavaScript and cookies were required by the site so Selenium and Playwright, Python libraries, were used.

In addition, the list of players from the original dataset was not complete. There were many players in the match dataset that were not originally accounted for. Therefore, the site was scraped for player names and IDs from those matches and our scraper for player statistics was run again. This provided us with in depth statistics for approximately 3681 players.

3.2 Preprocessing

Most data was provided in CSVs, making direct use simple. Many values, such as dates and some statistics, needed to be converted from strings to number values. For Handedness, Surface, Round, and Country (nominal values) one hot vectors were created. Win rates and days since last match were also manually calculated with statistics provided in the main dataset.

3.3 Annotation

No annotation was needed. Match wins were provided along with the match statistics from the main dataset.

3.4 Properties

After preprocessing, we were left with $95,773$ total matches, between 3681 different players. This is an increase from the $93,622$ total matches and 683 players used in our previous report. Most of these matches were previously missing player statistics. This gives us a total of $189,408$ feature vectors. Each match has 55 features, 51 being numeric and 6 being nominal. Note that all statistics are calculated for before the match being predicted.

We are using data from 1991-2017; however, due to world events, data is unevenly distributed throughout these years.

Exactly 50% of the matches are won by Player A and the rest by Player B, meaning the dataset is well distributed between each class, reflecting real-world applications of the model. Rounds and surface distributions are found in the following tables:

Surface	% of Total Matches
Hard	51.34
Clay	32.33
Grass	11.01
Carpet	5.32

Figure 1: Distribution of Surfaces

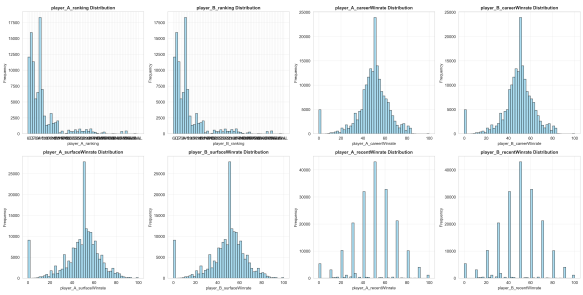

Figure 3: Distribution of Rankings and Win Rates

Round	% of Total Matches
Round of 32	30.10
Round of 16	15.53
Round of 64	14.08
Round of 128	8.44
Quarter-Finals	7.81
Semi-Finals	3.95
Finals	2.00
1st Round Qualifying	10.45
2nd Round Qualifying	5.56
3rd Round Qualifying	1.93
Round Robin	0.14
Olympic Bronze	0.001

Figure 2: Distribution of Rounds

Finally, the distribution of rankings and win rates can be found in below:

3.5 Updates to the Dataset

Unfortunately, we were not able to scrape additional match data for years after 2017. However, as mentioned above, we were able to fill the gap between our player statistics and match data meaning less sparsity in our dataset. Previously, approximately 15% of the matches were missing values for 20 of their features.

4 Features and Inputs

Our model uses 55 features, 51 of which are numerical and 6 are nominal. Some examples of our inputs are win-rates (on surface, in their career, and their recent win-rate for the past 10 games), the service statistics for a given player, break points for a given player, their recent match statistics, their attributes such as height, country, handedness, to name a few, and the match info. These are all key factors that might play into the outcome of a tennis match, and that's why we've decided to consider them for our model.

The feature engineering we performed on our data is that we needed to filter all matches that occurred before the current match being considered, and calculating all of the statistics/features using only this historical data. By considering only the historical data, we prevent any data leakage from happening. As an example, features calculated from historical data are win-rates (career, surface, head-to-head, last ten games), and their recent match information (win-streaks, days since last match, serving statistics, break points statistics, total points). For example, player_A_surfaceWinrate for a clay court match is calculated using only the player's previous clay court matches, not including the current match. This prevents the model from being able to make an inference on a current match based on future results.

Our categorical features were converted into 8-dimensional vectors rather than one-hot vectors, allowing the model to calculate relationships between some of our categorical features. For example, a hard court and a clay court hold more similarities than a grass court, and this should be considered when classifying. Our numerical features are standardized with normalization.

When experimenting with a Random Forest model, we utilized the same features and inputs. We found that, as expected, player rankings, surface win-rates, and career win-rates were found to be the most important features after training our random forest model.

For our feature selection, we considered all of our features and didn't leave any out.

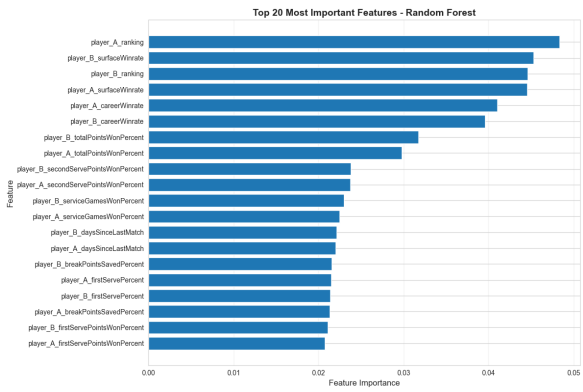

Figure 4: Ranking of Features by Importance for Random Forest

5 Implementation

Our model originally implemented a convolutional neural network with an input layer, three convolutional layers with 64, 128 and 256 filters. Each convolutional layer is followed by batch normaliza-

tion, ReLU activation, max pooling, and dropout with a dropout rate of 0.2. We then have three fully connected layers with 512, 256, and 128 nodes. Each fully connected layer contains batch normalization followed by ReLU, with a dropout rate of 0.3. We then go through our output layer, which applies the sigmoid function, producing the probability of player A winning. For the loss function of this convolutional neural network we used binary cross-entropy loss. We then used early-stopping Stochastic Gradient Descent with a learning rate of 0.001 and a batch size of 64 samples to train our model. However, after looking at the set of features, we realized this implementation wouldn't produce the most accurate result, as we don't exactly have a one-dimensional image with our tennis match data.

After further experimentation, we decided to implement a fully connected neural network with three fully connected layers the first fully connected layer would take our 41 numeric and 8 categorical features and output 512 neurons, then another layer outputting 256 neurons, and our last layer outputting 128 neurons. Each of these layers were followed by batch normalization and ReLU activation with a dropout rate of 0.3. We then had a final output layer producing a single neuron with sigmoid activation to give us the probability of Player A winning the given match. We also used binary cross-entropy loss for this neural network as well.

Additionally, we experimented with a random forest model. After hyper parameter tuning we decided that the best results were achieved with an implementation that uses 200 trees, 10 minimum samples required to split a node, and a minimum of four samples required at each leaf node. For the loss function we used Gini impurity.

Lastly, we experimented slightly with ensemble models of the above models we used with the idea that better performing models will get more influence, but perhaps there would be some benefit to certain games or match features being considered by other models. For this, we looked at simple majority voting, weighted voting by validation preformance, simple probability averaging, and stacking with logistic regression meta-learning. We only produced a simple version of all of the above ensemble models to attempt to increase the validation accuracy.

6 Evaluation Strategy

For the evaluation strategy, many methods were considered. However, due to the complexity of the model itself and the large amount of features, a simple evaluation strategy was chosen to fit with our available computation power; Validation splits. Initially, we intended to train on all data prior to 2015, validate on 2016 matches, and test on 2017 matches. It would be fed into the model in chronological order. This was to ensure that no data leakage occurred, making sure the model only trained on data prior to its current match, preventing it from predicting game results with knowledge it was not supposed to have. However, we instead chose to calculate all match stats in pre-processing, ensuring that all of the stats required for a prediction are calculated up until the game it is predicting. For example, a game in 2001 with Player A will have all of Player A's stats up until that game. This allowed us to use the more reliably tested method of random validation splits with a 10% allocation to validation, and a 20% allocation to testing.

Our primary metric for evaluation will be logistic loss, as it rewards predictions based on how close they were to being correct, not just based on correctness. This was inspired by Seranno (2020), but also made the most sense for the context of sports prediction. Log Loss incentivizes accuracy within predictions, not just correct predictions. As it is impossible to always correctly predict a sports game, it makes sense to encourage closer predictions instead.

Most simple baselines give the same number in our case: 50%. To start, we have the random baseline. Given that either player has a chance to win, this comes out to a 50%. As mentioned prior, the majority baseline provides the exact same 50% result. Therefore, our baseline is a 50% accuracy, with a goal of being closer to the 68.2% accuracy from Seranno (2020).

Evaluation Metric	Result
Test Loss	0.6366
Accuracy	0.6370%
Precision	0.6028
F1 Score	0.6241

Figure 6: Metrics to evaluate the quality of our model.

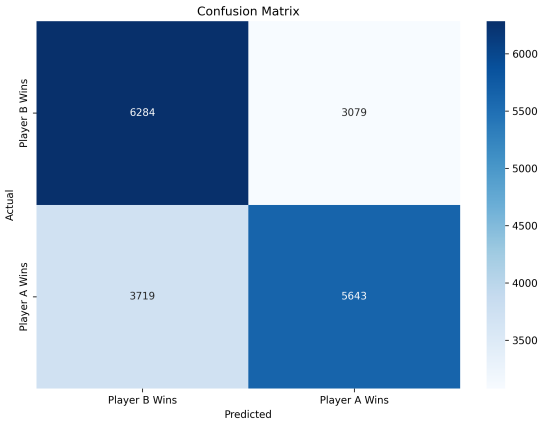

Figure 5: The confusion matrix produced by our model.

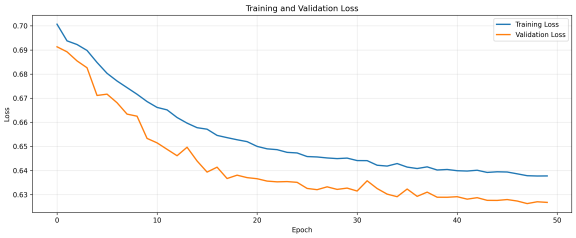

Figure 6: The training loss and validation loss curves of our model from 50 epochs.

7 Progress

As suggested by the TA following our progress report, we switched to an Adam optimizer instead of SGD. This did lead to an increase in accuracy as the Adam optimizer is better at generalizing. As mentioned previously in Section 3.5, we did scrape additional player statistics to fill the 15% gap in our dataset. So our dataset is now much less sparse, also leading to the increase in accuracy.

We did attempt to scrape additional data from matches after 2017; however, due to limited time and computing resources, we were unable to do so. The scrapers from the original dataset did not work.

In terms of our model, we have tested various hyperparameters using a larger grid search. We additionally attempted alternative models such as a fully connected neural network. The FCNN, in comparison to our previous convolutional neural network, did have an increased accuracy. Our previous model had a test accuracy of 63.70% where initial tests of the FCNN had a test accuracy of 66.49% before hyperparameter tuning. After hyperparameter tuning, we saw a test accuracy of 66.60%

Finally, the TA suggested we include additional

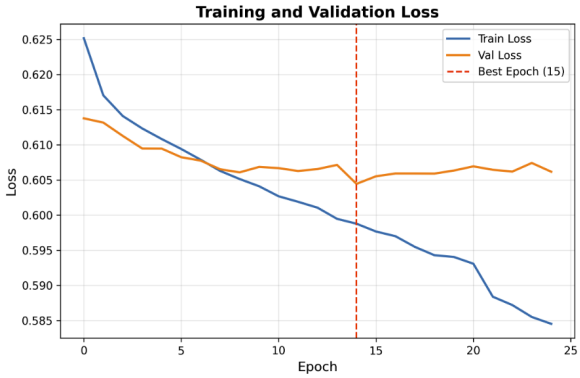

Figure 7: The training and validation loss curves of our fully connected neural network.

graphs and tables representing our experimental results; therefore, we have done so in this report.

8 Results and Error Analysis

Our initial CNN attempt was as follows: running our model on 93,622 matches with a 70% train, 10% validation, and 20% test split, we got a test accuracy of 63.70%. Well, this was above our baseline, we aimed to improve our hyperparameters and even make other changes to our model to improve our accuracy to get closer to the goal of 68.2% accuracy from Seranno (2020).

To examine the error of the models we've experimented with we looked at training and validation loss curves for the fully connected neural network, the confusion matrices produced by all of the models we've experimented with and the comparison of the validation accuracy percentage and F1-scores. Our model seemed to be generally evenly split when comparing the false positives percentage to false negatives percentage. The performance of our models are all relatively similar. However, there was a slight boost from the stacking ensemble model compared to the random forest and fully connected neural network on their own.

If we were to continue working on this model perhaps better an adjustment to the feature engineering would help improve the poor classification on tennis matches where one player is missing certain values, due the inconsistency of the ATP data. Additionally, adding more complexity to our fully connected layers of our FCNN, or adding more complexity to our random forest and ensemble methods could help improve the accuracy even more. Perhaps adding a logistic regression model to the ensemble implementation may also provide a short improvement (De Seranno).

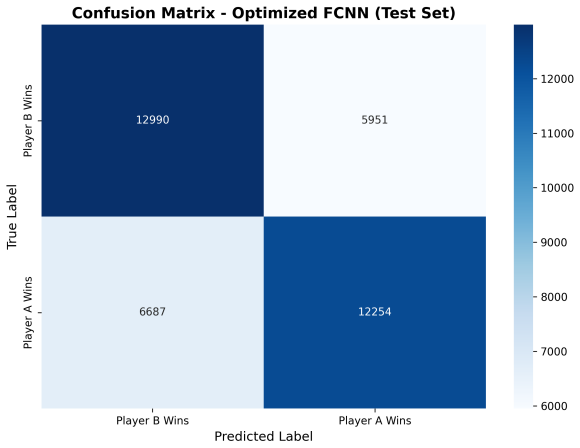

Figure 9: The confusion matrix produced by our FCNN.

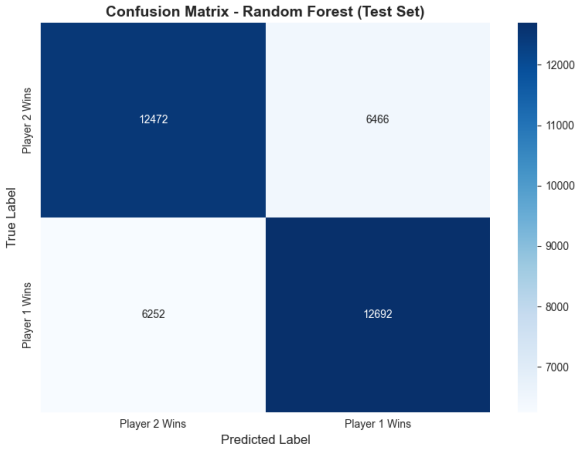

Figure 10: Confusion matrix produced by our Random Forest model

Model	Accuracy (%)	F1-Score
Random Forest	66.58	0.665
FCNN	66.24	0.653
Majority Vote	66.41	0.683
Weighted Vote	66.71	0.663
Prob. Averaging	66.72	0.664
Stacking	66.79	0.664

Figure 8: Comparison of Accuracy and F1-Score of Different Models.

Team Contributions

Leiah developed and ran the scraper for player statistics. She also parsed and preprocessed this data. Finally, she developed the initial FCNN model. Regarding the report, she wrote Sections 3 and 7.

Nic handled feature engineering and implemented the CNN and random tree models. He also wrote Sections 4, 5 and a small part of 6 of this report.

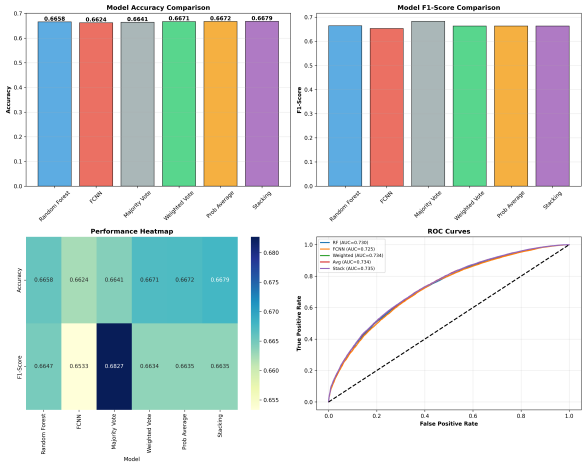

Figure 11: The ensemble comparisons of different models experimented with

Martin implemented the grid search for hyperparameter tuning. He also did large amounts of research into the related work, figuring out our baseline and goals. He provided feedback and support on model training and creation as well. Regarding the report, he wrote Sections 1, 2, and parts of 6.

References

ATP Tour. n.d. Stats: Atp tour: Tennis. https://www.atptour.com/en/stats/stats-home.

Green Code. 2025. I trained ai to predict sports. YouTube Video - https://www.youtube.com/watch?v=LkJpNLIaeVk.

DataHub. n.d. Atp world tour tennis data. https://datahub.io/core/atp-world-tour-tennis-data.

Mateus de Araujo Fernandes. 2017. Using soft computing techniques for prediction of winners in tennis matches. *Machine Learning Research*, 2(3):86–98.

Jingdi Lei, Tianqi Kang, Yuluan Cao, and Shiwei Ren. 2024. Capturing momentum: Tennis match analysis using machine learning and time series theory. *Preprint*, arXiv:2404.13300.

Tianyu Luo, Peizhe Zhao, and Xu Cheng. 2025. Tennis match prediction model based on deep learning and multi-source information fusion. In *Proceedings of the 2025 5th International Conference on Automation Control, Algorithm and Intelligent Bionics*, ACAIB '25, page 246–250, New York, NY, USA. Association for Computing Machinery.

Milkpainter. 2025. ultimate-tennis-predictor. Repo - https://github.com/Milkpainter/ultimate-tennis-predictor/tree/main.

Bai Rui, Chong Kar Hing, Li Haoyuan, and Teh Jia Yew. 2024. Forecasting tennis player matches based on machine learning. In *Proceedings of 2024 International Conference on Machine Learning and Intelligent Computing*, volume 245 of *Proceedings of Machine Learning Research*, pages 393–403. PMLR.

Alexander De Seranno. 2020. Predicting tennis matches using machine learning. PDF - https://libstore.ugent.be/fulltxt/RUG01/002/945/727/RUG01-002945727_2021_0001_AC.pdf.